A Kleisli Category Model of AI Agent Behavior

Dmitry Kolesnikov · `github.com/kshard/thinker` · April 2026

Abstract

Large-language-model (LLM) agents are typically described informally: a bot receives a prompt, calls tools, and produces a reply. This paper gives a categorical account of that intuition. We show that every agent behaviour — sequential composition, reflective self-correction, and plan-then-execute — can be expressed as a morphism in the **Kleisli category** of the combined error-and-state monad. The resulting algebra has two primitive morphisms (ReAct and Pure), four bridging concepts (Arr, Arrow, Eff, Hoist), and three structural combinators (Seq, Reflect, ThinkReAct) that correspond directly to well-known patterns in multi-agent system design.

1. Introduction

The practical engineering of LLM-backed systems has outpaced its theoretical foundations. Frameworks accumulate ad-hoc combinators — chains, retries, map-reduce sweeps — without a common algebraic language that would explain why some compositions are valid and others are not, or predict how behaviours compose.

Category theory has long served as the mathematics of composition. The Kleisli construction, in particular, provides a canonical way to reason about effectful programs: it turns a monad into a category whose morphisms are *programs with effects*, and in which composition is exactly monadic bind ($\gg\!=$). This paper applies that machinery to LLM agents operating over a shared mutable *blackboard* — a typed state record S that accumulates knowledge base.

The central claim is:

> ***An LLM agent step is a Kleisli endomorphism $S \twoheadrightarrow S$ on the blackboard. Agent composition is Kleisli composition. All standard multi-agent patterns are instances of this single algebraic structure.***

2. Background

2.1 The Error Monad with Context

Let Ctx be the abstract execution context. Define the base monad:

$$M\,A \triangleq \mathrm{Ctx} \to \mathrm{Either}(\varepsilon,\, A)$$

This is the IO-monad restricted to errors, with a Reader component for Ctx. Monadic return and bind are:

$$\eta_A(a) \triangleq \lambda\gamma.\ \mathrm{Right}(a)$$

$$(m \gg\!= k) \triangleq \lambda\gamma.\ \mathrm{case}\ m(\gamma)\ \mathrm{of}\ \mathrm{Left}(e) \Rightarrow \mathrm{Left}(e) \mid \mathrm{Right}(a) \Rightarrow k(a)(\gamma)$$

Bind sequences two computations, short-circuiting on the first error.

2.2 The State Monad Transformer

Lifting M with a state component S gives:

$$\mathrm{StateT}\,S\,M\,A \;=\; S \to M\,(A \times S)$$

A value of type $\mathrm{StateT}\,S\,M\,A$ is a program that reads a blackboard S, may fail, and returns both a result of type A and an updated blackboard. modify, get, and put are the standard primitives.

2.3 The Kleisli Category

Given a monad $(M, \eta, \gg=)$, the **Kleisli category** $\mathbf{Kl}(M)$ has:

- **objects**: types
- **morphisms** $A \twoheadrightarrow B$: functions $A \to M\,B$
- **identity** on A: $\eta_A : A \to M\,A$
- **composition** $(f : A \twoheadrightarrow B) \gg\!\!> (g : B \twoheadrightarrow C) : A \twoheadrightarrow C$:

$$x \mapsto f(x) \gg= g$$

Composition is associative and respects the identity laws — this is exactly what the monad laws guarantee.

2.4 Lenses as Bidirectional Accessors

A **lens** from S to A is a pair $(\mathrm{get}, \mathrm{put})$ where $\mathrm{get} : S \to A$ extracts a component and $\mathrm{put} : S \times A \to S$ replaces it [4, 5]. The put half is the algebraic structure that this paper requires: it makes S a carrier for the action of A, folding an agent's output back into the blackboard. Curried, $\mathrm{put}(a) : S \to S$ is the action of command a on S.

When S is a product type (a record), the lens for each field are derived automatically — ensuring that the state monad transformer is consistent with the blackboard's layout.

3. The Agent Algebra

3.1 The Bot Morphism

The primitive abstraction is the **Bot** — a morphism in $\mathbf{Kl}(M)$:

$$\mathrm{Bot}[S, A] \triangleq S \to M\,A$$

It reads the current blackboard S, performs LLM reasoning (including all possible side-effectes), and returns a typed result A. Two type parameters are necessary: the input type S (the blackboard) and the output type A (the result, which is generally different from S).

3.2 ReAct: The Primitive Realisation of Bot

The definition of $\mathrm{Bot}[S, A]$ is abstract — it specifies the type of a Kleisli morphism without prescribing how the LLM interaction is carried out. The **ReAct** (Reason-and-Act) pattern provides the canonical computational realisation.

A ReAct agent encapsulates a cyclic process of *thinking* (LLM reasoning), *acting* (tool invocation — side effect), and *observing* (feeding tool results back into the context) that repeats until the model produces a final answer. Internally, the agent threads a *memory* — an ordered sequence of observations — and an *encoder/decoder* pair that mediates between the typed input A / output B and the LLM's textual protocol. Externally, the entire cycle is collapsed into a single invocation:

$$\mathrm{ReAct}[A, B] \;:\; \mathrm{Bot}[A, B]$$

The internal loop is defined as a least fixed point over the LLM reply stage. Let Mem be the memory state, $\mathrm{enc} : A \to M\,\mathrm{Prompt}$ the encoder, $\mathrm{dec} : \mathrm{Reply} \to M\,B$ the decoder, $\mathrm{llm} : \mathrm{Prompt} \times \mathrm{Mem} \to M\,(\mathrm{Reply} \times \mathrm{Mem})$ the model call, and $\mathrm{tools} : \mathrm{Reply} \to M\,\mathrm{Prompt}$ the tool-dispatch oracle. Then:

$$\mathrm{ReAct}(a) \triangleq \mathrm{enc}(a) \gg= \mathrm{loop}$$

$$\text{loop}(p) \triangleq \text{llm}(p) \gg\!= \lambda(r, m').\ \begin{cases} \text{dec}(r) & \text{if } r.\text{stage} = \text{RETURN} \\ \text{tools}(r) \gg\!= \text{loop} & \text{if } r.\text{stage} = \text{INVOKE} \\ \text{Left}(\varepsilon_{\text{aborted}}) & \text{otherwise} \end{cases}$$

Because the loop terminates on RETURN or on error, and external timeout is enforced by Ctx, ReAct is a well-defined Kleisli morphism $A \twoheadrightarrow B$.

Lemma 3.2 (ReAct is a Bot). $\text{ReAct}[A, B]$ *satisfies* $\text{Bot}[A, B]$*: it is a function* $A \to M\,B$.

Proof. $\text{enc} : A \to M\,\text{Prompt}$, $\text{loop} : \text{Prompt} \to M\,B$ (by case analysis on the reply stage, each branch returns $M\,B$). Their Kleisli composition $\text{enc} \gg\!\!> \text{loop} : A \to M\,B$. □

The role of ReAct in the algebra is foundational: it is the only combinator that actually invokes an LLM. Every other combinator in the hierarchy (Arrow, Seq, Reflect, ThinkReAct) is a *structural* composition that takes one or more Bot values and produces a new Bot. The ReAct instances are the leaves of every composition tree.

3.3 Pure: The Deterministic Realisation of Bot

Not every Bot requires an LLM. Pure lifts an ordinary Kleisli morphism $f : S \to M\,A$ into $\text{Bot}[S, A]$:

$$\text{Pure} : (S \to M\,A) \to \text{Bot}[S, A]$$

$$\text{Pure}(f) \triangleq f$$

Where ReAct is the *non-deterministic* primitive (it calls an external LLM), Pure is its *deterministic* counterpart: it performs a computation that depends only on the blackboard and the execution context. Typical uses include computing a derived value from S.

Lemma 3.3 (Pure is a Bot). $\text{Pure}(f)$ *satisfies* $\text{Bot}[S, A]$ *for any* $f : S \to M\,A$.

Proof. $\text{Bot}[S, A] \triangleq S \to M\,A$. Since $f : S \to M\,A$, the inclusion is the identity. □

Together ReAct and Pure exhaust the two modes of Bot construction: LLM-backed and deterministic. Every structural combinator (Arrow, Seq, Reflect, ThinkReAct, Think) composes over Bot values regardless of which primitive produced them.

3.4 From Bot to Arrow: The Bridging Problem

A single ReAct agent produces a result of type A, but the blackboard has type S. To compose agents into a complex task solver we need *endomorphisms* $S \to M\,S$, not heterogeneous morphisms $S \to M\,A$. The gap between $\text{Bot}[S, A]$ and $\text{Arr}[S]$ is bridged by an *effect* — a description of how to fold a bot's output back into the blackboard and then post-process the result.

$\text{Eff}[S, A]$ is the composite effect that Arrow applies. It has two components:

$\text{Lens}[S, A]$ is the put half of a lens (§2.4): it folds the agent's output A back into the blackboard. Curried, it is $A \to (S \to S)$, a *left action* of A on S. When S is a product type the lens is derived automatically by structural reflection.

$$\text{Lens}[S, A] \triangleq S \times A \to S$$

$\text{Eval}[S]$ is a side-effectful hook that runs *after* the lens has been applied — persistence, validation, guardrails. Its type coincides with that of $\text{Arr}[S]$ (both are $S \to M\,S$), but its *role* is different: Eval is a single post-processing step, whereas Arr is the composite agent that includes the bot call, the lens, and the eval.

$$\text{Eval}[S] \triangleq S \to M\,S$$

The effect bundles both into a single record. When Lens is absent the lens is derived automatically via structural reflection; when Eval is absent the identity η_S is used.

$$\mathrm{Eff}[S, A] \triangleq \mathrm{Lens}[S, A] \times \mathrm{Eval}[S]$$

3.5 The Kleisli Arrow: Arr and Arrow

With the bridging types in hand, we define the *Kleisli arrow* — the endomorphism that every agent step must inhabit — this abstraction enforces determisism for probabilistic Bot behaviour:

$$\mathrm{Arr}[S] \triangleq S \to M\,S$$

Lemma 3.4 (Arr–Bot Isomorphism). $\mathrm{Arr}[S] \cong \mathrm{Bot}[S, S]$.

Proof. Both are defined as $S \to M\,S$. The isomorphism is the identity function: given $f : \mathrm{Arr}[S]$, define $\mathrm{Prompt}(f)(s) \triangleq f(s)$; conversely, given $b : \mathrm{Bot}[S, S]$, define $f(s) \triangleq b(s)$. □

This isomorphism makes every arrow directly usable wherever a bot is expected — the algebra is *closed*.

Arrow is the canonical lift of $\mathrm{Bot}[S, A]$ into $\mathrm{Arr}[S]$. It consumes the bridging Eff to absorb the output type A:

$$\mathrm{Arrow} : \mathrm{Bot}[S, A] \times \mathrm{Eff}[S, A] \to \mathrm{Arr}[S]$$

$$\mathrm{Arrow}(b, \varphi) \triangleq \lambda s.\ b(s) \gg\!= \lambda a.\ \varphi.\mathrm{Eval}(\varphi.\mathrm{Lens}(s,\ a))$$

The two steps are:

1. **Lens put** (pure): $\varphi.\mathrm{Lens}(s,\ a)$ folds the bot's output into the blackboard — a morphism in the ordinary category of types.
2. **Kleisli composition** (effectful): $\varphi.\mathrm{Eval}(s')$ applies the side-effect — a morphism in $\mathbf{Kl}(M)$.

Together they yield a single Kleisli endomorphism. Arrow is thus a mapping from the hom-set $\mathrm{Bot}[S, A]$ into $\mathrm{End}_{\mathbf{Kl}(M)}(S)$, the monoid of endomorphisms on S.

Lemma 3.5 (Arrow Well-Definedness). *For any $b : \mathrm{Bot}[S, A]$ and $\varphi : \mathrm{Eff}[S, A]$, $\mathrm{Arrow}(b, \varphi)$ is a valid Kleisli endomorphism $S \twoheadrightarrow S$.*

Proof. By the typing of the constituents. $b : S \to M\,A$. Given $s : S$, $b(s)$ yields $M\,A$. In the success branch, $\varphi.\mathrm{Lens}(s, a) : S$ since $\mathrm{Lens} : S \times A \to S$. Then $\varphi.\mathrm{Eval} : S \to M\,S$ applied to this result yields $M\,S$. Composing via bind:

$$\mathrm{Arrow}(b, \varphi)(s) \;=\; b(s) \gg\!= \lambda a.\ \varphi.\mathrm{Eval}(\varphi.\mathrm{Lens}(s, a))$$

which has type $M\,S$. Therefore $\mathrm{Arrow}(b, \varphi) : S \to M\,S = \mathrm{Arr}[S]$. □

3.6 Lifting Computations: Lift

Not every task solving step involves an LLM call. Pure computations, validation, or transformation steps that operate directly on the blackboard are plain functions $S \to M\,S$. The Lift combinator injects them into the Kleisli arrow type so they compose uniformly with bot-backed steps:

$$\mathrm{Lift} : \mathrm{Eval}[S] \to \mathrm{Arr}[S]$$

$$\mathrm{Lift}(e) \triangleq e$$

Lemma 3.6 (Lift Embedding). Lift *is a faithful embedding of* $(\mathrm{Eval}[S], \rightarrowtail)$ *into* $(\mathrm{Arr}[S], \rightarrowtail)$.

Proof. Lift is the identity on the underlying function type. Faithfulness (injectivity) is immediate: $\mathrm{Lift}(e_1) = \mathrm{Lift}(e_2) \implies e_1 = e_2$. Composition is preserved because $\mathrm{Arr}[S]$ and $\mathrm{Eval}[S]$ share the same underlying Kleisli composition. □

Lift is the formal mechanism by which deterministic code (guardrails, enrichment, persistence) enters the Kleisli algebra on equal footing with LLM-backed agents.

3.7 Output Functor: Map

$$\mathrm{Map} : \mathrm{Bot}[S, A] \times (S \times A \to B) \to \mathrm{Bot}[S, B]$$

$$\mathrm{Map}(b, f) \triangleq \lambda s.\ b(s) \gg\!= \lambda a.\ \eta(f(s, a))$$

The function $f : S \times A \to B$ has access to both the original blackboard and the bot's output, enabling projections. The mapping $\mathrm{Bot}[S, -]$ defines a functor $\mathbf{Type} \to \mathbf{Type}$ with Map as its action on morphisms, parameterised over the reader S.

Lemma 3.7 (Map Identity). *Let $\pi_2 : S \times A \to A$ be the second projection. Then $\mathrm{Map}(b, \pi_2) = b$.*

Proof. For any $s : S$: $\mathrm{Map}(b, \pi_2)(s) = b(s) \gg\!= \lambda a.\ \eta(\pi_2(s, a)) = b(s) \gg\!= \lambda a.\ \eta(a) = b(s) \gg\!= \eta = b(s)$ by the monad right-identity law. □

Lemma 3.8 (Map Composition). *For $g : S \times A \to B$ and $f : S \times B \to C$, define the S-indexed composition $(f \bullet g)(s, a) \triangleq f(s,\ g(s, a))$. Then:*

$$\mathrm{Map}(\mathrm{Map}(b, g),\ f) \ =\ \mathrm{Map}(b,\ f \bullet g)$$

Proof. For any $s : S$:

$$\mathrm{Map}(\mathrm{Map}(b, g), f)(s) = \mathrm{Map}(b, g)(s) \gg\!= \lambda c.\ \eta(f(s, c))$$

Expanding the inner term:

$$= (b(s) \gg\!= \lambda a.\ \eta(g(s, a))) \gg\!= \lambda c.\ \eta(f(s, c))$$

By monad associativity:

$$= b(s) \gg\!= \lambda a.\ (\eta(g(s, a)) \gg\!= \lambda c.\ \eta(f(s, c)))$$

By left identity ($\eta(x) \gg\!= k = k(x)$):

$$= b(s) \gg\!= \lambda a.\ \eta(f(s,\ g(s, a))) = \mathrm{Map}(b,\ f \bullet g)(s) \quad \square$$

3.8 Cross-Type Lifting: Hoist

Arrow bridges $\mathrm{Bot}[S, A]$ and $\mathrm{Arr}[S]$ within a single blackboard type. A complementary problem arises when an existing $\mathrm{Arr}[T]$ pipeline — built for a different blackboard T — must be embedded into $\mathrm{Arr}[S]$. This is the *cross-type lifting* problem: how to reuse a finished sub-pipeline without duplicating it.

The solution is a *partial isomorphism* between S and T, defined by four lenses that share exactly two fields — one for input, one for output:

$$\mathrm{BiMap}[S, A, T, B] \triangleq (\ell_{SA},\ \ell_{TA},\ \ell_{TB},\ \ell_{SB})$$

Hoist uses this isomorphism to lift $\mathrm{Arr}[T]$ into $\mathrm{Arr}[S]$:

$$\text{Hoist} : \text{BiMap}[S, A, T, B] \times \text{Arr}[T] \to \text{Arr}[S]$$

$$\text{Hoist}(\text{iso}, f)(s) \triangleq \eta(\ell_{TA}.\text{put}(\mathbf{0}_T,\ \ell_{SA}.\text{get}(s))) \gg\!= \lambda t.\ \eta(\ell_{SB}.\text{put}(s,\ \ell_{TB}.\text{get}(t)))$$

where $\mathbf{0}_T$ is the zero-value allocation of type T. The three steps are:

1. **Inject** (pure): allocate $\mathbf{0}_T$ and copy S's field A into it via ℓ_{TA}. The rest of T is zero-initialised.
2. **Run** (effectful): apply the inner arrow $f : \text{Arr}[T]$ to the constructed T, which may perform LLM calls, tool invocations, or any composed sub-pipeline.
3. **Project** (pure): read field B from the resulting t via ℓ_{TB} and write it back into the original s via ℓ_{SB}. All other fields of S are preserved.

Lemma 3.9 (Hoist Well-Definedness). *For any* $\text{iso} : \text{BiMap}[S, A, T, B]$ *and* $f : \text{Arr}[T]$, $\text{Hoist}(\text{iso}, f) : \text{Arr}[S]$.

Proof. $f : T \to M\ T$. The inject step produces a value of type T purely. Then f applied to it yields $M\ T$. In the success branch, $\ell_{TB}.\text{get}(t) : B$ and $\ell_{SB}.\text{put}(s, \cdot) : B \to S$, so $\ell_{SB}.\text{put}(s, \ell_{TB}.\text{get}(t)) : S$. Wrapping with η yields $M\ S$. The overall composition has type $S \to M\ S = \text{Arr}[S]$. □

Lemma 3.10 (Hoist Preserves S). *All fields of* $s : S$ *except the* B*-component are unchanged by* $\text{Hoist}(\text{iso}, f)(s)$.

Proof. The project step applies only $\ell_{SB}.\text{put}$, which modifies exactly the B-field of s by the lens put-put law. Every other component of s is carried unchanged from the inject step. □

Hoist is the mechanism for reusing sub-pipelines across differently-shaped blackboards without code duplication. Because its result type is exactly $\text{Arr}[S]$, it composes freely with Seq, Reflect, and ThinkReAct.

4. Compositional Patterns

4.1 Sequential Composition: Seq

$$\text{Seq} : \text{Arr}[S]^* \to \text{Arr}[S]$$

$$\text{Seq}(f_1, f_2, \ldots, f_n) \triangleq f_1 \gg\!\!> f_2 \gg\!\!> \cdots \gg\!\!> f_n$$

Each step receives the blackboard produced by the previous step; the first error short-circuits. The identity element is $\eta_S : S \to M\ S$.

The caller composes independently-typed bots by first lifting each with Arrow to absorb the output type parameter:

$$\text{Seq}(\text{Arrow}(b_A : \text{Bot}[S, A],\ \varphi_A),\ \text{Arrow}(b_B : \text{Bot}[S, B],\ \varphi_B)) \ :\ \text{Arr}[S]$$

The intermediate types A and B disappear — absorbed into the blackboard by Eff.Lens. This is the key advantage of the Kleisli formulation: it enables *type-safe erasure of intermediate result types* without losing information.

State threading diagram:

$$S_0 \xrightarrow{f_1} S_1 \xrightarrow{f_2} S_2 \xrightarrow{\cdots} S_n$$

Lemma 4.1 (Seq Associativity). *For any* $f, g, h : \text{Arr}[S]$*:*

$$\text{Seq}(f, \text{Seq}(g, h)) \ =\ \text{Seq}(\text{Seq}(f, g), h) \ =\ \text{Seq}(f, g, h)$$

Proof. Seq is defined as iterated Kleisli composition ($\gg\!\!>$). Unfolding:

$$\text{Seq}(f, \text{Seq}(g, h))(s) \ =\ f(s) \gg\!= (\lambda s_1.\ g(s_1) \gg\!= h)$$

$$\mathrm{Seq}(\mathrm{Seq}(f, g), h)(s) \;=\; (f(s) \gg\!= g) \gg\!= h$$

By the **monad associativity law**, $(m \gg\!= f) \gg\!= g = m \gg\!= (\lambda x.\, f(x) \gg\!= g)$, these two expressions are identical. □

Lemma 4.2 (Seq Identity). *Let* $\mathrm{id}_S \triangleq \eta_S$. *Then for any* $f : \mathrm{Arr}[S]$*:*

$$\mathrm{Seq}(\mathrm{id}_S, f) \;=\; f \;=\; \mathrm{Seq}(f, \mathrm{id}_S)$$

Proof. Left identity: $\mathrm{Seq}(\eta_S, f)(s) = \eta_S(s) \gg\!= f = f(s)$ by the monad left-identity law ($\eta(a) \gg\!= k = k(a)$).

Right identity: $\mathrm{Seq}(f, \eta_S)(s) = f(s) \gg\!= \eta_S = f(s)$ by the monad right-identity law ($m \gg\!= \eta = m$). □

Corollary 4.3 (Endomorphism Monoid). $(\mathrm{Arr}[S], \mathrm{Seq}, \eta_S)$ *forms a monoid — the endomorphism monoid* $\mathrm{End}_{\mathbf{Kl}(M)}(S)$.

Lemma 4.4 (Arrow–Seq Coherence). *Lifting commutes with sequencing:*

$$\mathrm{Seq}(\mathrm{Arrow}(b_1, \varphi_1),\ \mathrm{Arrow}(b_2, \varphi_2)) \;=\; \mathrm{Arrow}(b_1, \varphi_1) \gg\!\!> \mathrm{Arrow}(b_2, \varphi_2)$$

Proof. Immediate from the definition of Seq as iterated $\gg\!\!>$. □

Lemma 4.5 (Lift–Seq Homomorphism). Lift *preserves Kleisli composition:*

$$\mathrm{Seq}(\mathrm{Lift}(e_1),\ \mathrm{Lift}(e_2)) \;=\; \mathrm{Lift}(e_1 \gg\!\!> e_2)$$

Proof. By Lemma 3.6, Lift is the identity on the underlying function, so $\mathrm{Seq}(\mathrm{Lift}(e_1), \mathrm{Lift}(e_2)) = e_1 \gg\!\!> e_2 = \mathrm{Lift}(e_1 \gg\!\!> e_2)$. □

4.3 Verdict and Judge: Vote and Judge

Before defining the reflection loop we introduce its decision type.

$$\mathrm{Vote}[S] \;\triangleq\; S \times \{-1,\ 0,\ +1\}$$

A value (s', v) carries the blackboard updated with judge feedback s' and the decision signal v:

v	Meaning	Behaviour
$+1$	**accept**	s' is returned as the final blackboard
0	**revise**	s' (carrying critique) forwarded to corrector
-1	**reject**	hard reject; error returned immediately

Judge lifts $\mathrm{Bot}[S, A]$ into $\mathrm{Bot}[S, \mathrm{Vote}[S]]$:

$$\mathrm{Judge} : \mathrm{Bot}[S, A] \times \mathrm{Eff}[\mathrm{Vote}[S],\ A] \to \mathrm{Bot}[S, \mathrm{Vote}[S]]$$

$$\mathrm{Judge}(b, \varphi) \;\triangleq\; \lambda s.\, b(s) \gg\!= \lambda a.\, \varphi.\mathrm{Eval}(\varphi.\mathrm{Lens}(\langle s, 0\rangle,\ a))$$

The default $\varphi.\mathrm{Lens}$ is the composed lens $\mathrm{Vote}[S].\mathrm{State} \circ \ell_{S,A}$ that writes A into the S field of Vote and leaves the signal at its zero value. The default $\varphi.\mathrm{Eval}$ is "always accept" ($v \mapsto +1$).

Judge separates the concern of *evaluating* content (the raw bot b) from the concern of *deciding* whether to accept (the fold φ), allowing the same underlying bot to serve different reflection policies.

4.4 Reflective Guard Loop: Reflect

$$\text{Reflect} : \text{Bot}[S, \text{Vote}[S]] \times \text{Arr}[S] \times \mathbb{N} \to \text{Bot}[S, S]$$

The reflection pattern implements a bounded guard/review loop. Formally:

$$\text{Reflect}(j, c, n)(s_0) \triangleq \text{iter}(n, s_0)$$

where iter is defined by structural recursion on the attempt counter:

$$\text{iter}(0, s) = \text{Left}(\varepsilon_{\text{exhausted}})$$

$$\text{iter}(k, s) = j(s) \gg\!= \lambda(s', v).\ \begin{cases} \eta(s') & \text{if } v = +1 \\ \text{Left}(\varepsilon_{\text{rejected}}) & \text{if } v = -1 \\ c(s') \gg\!= \lambda s''.\ \text{iter}(k{-}1,\ s'') & \text{if } v = 0 \end{cases}$$

The corrector $c : \text{Arr}[S]$ is a full Kleisli arrow — not a raw $\text{Bot}[S, B]$. This is the key structural property: the corrector's own Eff (lens + eval) is encapsulated at construction time via $\text{Arrow}(b_c, \varphi_c)$. The reflection loop is oblivious to the corrector's internal output type.

Because Reflect returns S in M, it satisfies $\text{Bot}[S, S]$ and is therefore isomorphic to $\text{Arr}[S]$ by Lemma 3.4. This allows nesting a reflection loop inside a Seq or another Reflect.

Lemma 4.6 (Reflect Termination). *For any j, c, and $n \in \mathbb{N}$, $\text{Reflect}(j, c, n)$ invokes the judge at most n times and the corrector at most $n - 1$ times.*

Proof. By structural induction on n. Base case ($n = 0$): no invocation, immediate error. Inductive step ($n = k + 1$): the judge is invoked once. On accept or reject the loop terminates (1 judge invocation, 0 corrector invocations). On revise, the corrector is invoked once and the recurrence $\text{iter}(k, s'')$ is entered, which by the inductive hypothesis invokes the judge at most k times and the corrector at most $k - 1$ times. Total: $1 + k = n$ judge invocations and $1 + (k - 1) = n - 1$ corrector invocations. □

Lemma 4.7 (Reflect Convergence). *If j always returns $v = +1$ (accept), then $\text{Reflect}(j, c, n) = j \gg\!= \pi_{\text{State}}$ for all $n \geq 1$, regardless of c.*

Proof. On the first iteration, $j(s)$ yields $(s', +1)$ and the loop returns $\eta(s')$. The corrector c is never invoked. □

4.5 Plan-and-Execute: ThinkReAct and Think

4.5.1 The Plan Combinator: Think

A planner typically returns a list of *task descriptors* $[A]$ that has to be elevated into blackboard per-task state T for the reactor to operate. Think applyies a *transform* function $\sigma : S \times A \to T$ to each element:

$$\text{Think} : \text{Bot}[S, [A]] \times (S \times A \to T) \to \text{Bot}[S, [T]]$$

$$\text{Think}(b, \sigma) \triangleq \lambda s.\ b(s) \gg\!= \lambda[a_1, \ldots, a_n].\ \eta([\sigma(s, a_1), \ldots, \sigma(s, a_n)])$$

The transform function projects the outer blackboard S and each task item A into the inner per-task state T.

Lemma 4.8 (Think Purity). $\text{Think}(b, \sigma)$ *invokes b exactly once and applies σ purely — no monadic effects are introduced beyond those of b itself.*

Proof. The outer η wraps the mapped list in the monad without effects. σ is a pure function. □

4.5.2 The Plan-and-Execute Combinator: ThinkReAct

$$\text{ThinkReAct} : \text{Bot}[S, [T]] \times \text{Arr}[T] \times (S \times [T] \to S) \to \text{Arr}[S]$$

ThinkReAct implements the plan-and-execute pattern. It is structurally *different* from Seq and Reflect: it operates across **two Kleisli categories** — the outer $\mathbf{Kl}(M)(S)$ and the inner $\mathbf{Kl}(M)(T)$ — connected by a unfold/fold pair.

The think bot $p : \mathrm{Bot}[S, [T]]$ produces the task list, where each element is already in the inner per-task state T (use Think to transform from $\mathrm{Bot}[S, [A]]$ when the planner's output type differs). The reactor $r : \mathrm{Arr}[T]$ is a full Kleisli arrow — a composed agent (Seq, Reflect, nested ThinkReAct) in the inner state space. The gather $\gamma : S \times [T] \to S$ folds the per-task results back into the outer state.

$$\mathrm{ThinkReAct}(p, r, \gamma)(s) \triangleq p(s) \gg\!= \lambda[t_1, \ldots, t_n].\ \eta(\gamma(s,\ [r(t_i) \mid i = 1 \ldots n]))$$

where each $r(t_i)$ is evaluated independently (sequentially or in parallel) and the results are collected into a list before applying γ.

When γ is omitted, the default is the auto-derived lens $\mathrm{Lens}[S, [T]]$, consistent with Arrow's default behaviour.

The gather fold absorbs the list of per-task results into the blackboard, making ThinkReAct a first-class citizen of the $\mathrm{Arr}[S]$. It composes directly in Seq, nests inside Reflect, and can even serve as the react arrow of an outer ThinkReAct.

This is a critical semantic distinction from Seq:

	Seq	ThinkReAct
State threading	Forward: $s_{i+1} = f_i(s_i)$	None: each task starts from t_i
State types	Single: S	Two: outer S, inner T
Reactor type	$\mathrm{Arr}[S]$	$\mathrm{Arr}[T]$ (composable agent)
Result type	S (single)	S (via fold)
Category	$\mathrm{End}_{\mathbf{Kl}(M)}(S)$ monoid	Cross-category traversal $S \to [T] \to S$

Lemma 4.9 (ThinkReAct Independence). *In* $\mathrm{ThinkReAct}(p, r, \gamma)(s)$*, the computation* $r(t_i)$ *does not depend on the result of* $r(t_j)$ *for* $i \neq j$.

Proof. Each task's computation $r(t_i)$ references only its own per-task state t_i. No variable from another task's computation appears in this expression. □

Corollary 4.10 (Parallelisability). *The tasks in* ThinkReAct *may be evaluated in any order or in parallel without affecting the result (up to non-determinism in* M*).*

Lemma 4.11 (ThinkReAct is Arr). $|text{ThinkReAct}(p, r, |gamma) : |text{Arr}[S]$.

Proof. $p : S \to M\,[T]$. The traversal maps each t_i through $r : T \to M\,T$ and collects into $M\,[T]$. Then $\gamma(s, -) : [T] \to S$ is pure, lifted into M via η. The overall composition $s \mapsto p(s) \mathbin{\gg!=} (\text{traverse}; r) \mathbin{\gg!=} \lambda ts.; \eta(\gamma(s, ts))$ has type $S \to M\,S = \mathrm{Arr}[S]$. □

5. The Full Hierarchy

$\mathrm{ReAct}[A, B]$ — primitive LLM agent (reason-act-observe loop)
$\mathrm{Pure}(f)$ — deterministic agent (no LLM call)
|
$\mathrm{Bot}[S, A]$ — Kleisli morphism $S \to M\,A$
| $\mathrm{Arrow}(\varphi)$ — absorbs A via Eff
$\mathrm{Arr}[S]$ — Kleisli endomorphism $S \twoheadrightarrow S \;\cong\; \mathrm{Bot}[S, S]$
|
⊢ $\mathrm{Seq}(f, g, \ldots)$ — monoid composition $f \gg\!\!> g \gg\!\!> \cdots$
|
⊢ $\mathrm{Reflect}(\mathrm{judge}, c, n)$ — bounded iteration with guard
| ⊢ $\mathrm{judge} : \mathrm{Bot}[S, \mathrm{Vote}[S]]$ — via $\mathrm{Judge}(b, \varphi)$, A absorbed
| ⊢ $c : \mathrm{Arr}[S]$ — corrector (Bot + Eff, B absorbed)
|
⊢ $\mathrm{ThinkReAct}(p, r, \gamma)$ — transform/fold traversal $S \to [T] \to S$
⊢ $p : \mathrm{Bot}[S, [T]]$ — planner (via $\mathrm{Think}(\sigma)$)
⊢ $r : \mathrm{Arr}[T]$ — reactor (composable agent)
⊢ $\gamma : S \times [T] \to S$ — gather fold

$\mathrm{Lift}(e)$ — embed $\mathrm{Eval}[S] \hookrightarrow \mathrm{Arr}[S]$
$\mathrm{Hoist}(\mathrm{iso}, f)$ — lift $\mathrm{Arr}[T] \to \mathrm{Arr}[S]$ via $\mathrm{BiMap}[S, A, T, B]$
$\mathrm{Judge}(b, \varphi)$ — lift $\mathrm{Bot}[S, A] \to \mathrm{Bot}[S, \mathrm{Vote}[S]]$
$\mathrm{Think}(b, \sigma)$ — transform $\mathrm{Bot}[S, [A]] \to \mathrm{Bot}[S, [T]]$
$\mathrm{Map}(b, f)$ — functor on output: $\mathrm{Bot}[S, A] \to \mathrm{Bot}[S, B]$

Every node in the upper hierarchy satisfies $\mathrm{Bot}[S, -]$. Composition can be arbitrarily nested. As an example, an agent with an inner reflection loop:

$$\mathrm{inner} \;\triangleq\; \mathrm{Arrow}(b_A,\ \varphi_1) \;:\; \mathrm{Arr}[S]$$

$$\mathrm{reflect} \;\triangleq\; \mathrm{Reflect}(\mathrm{Judge}(b_J, \varphi_J),\ \mathrm{inner},\ 3) \;:\; \mathrm{Bot}[S, S] \cong \mathrm{Arr}[S]$$

$$\mathrm{outer} \;\triangleq\; \mathrm{Seq}(\mathrm{reflect},\ \mathrm{Arrow}(b_B, \varphi_2)) \;:\; \mathrm{Arr}[S]$$

$$\mathrm{result} \;\triangleq\; \mathrm{Map}(\mathrm{outer},\ \lambda(s, s').\ \pi_B(s')) \;:\; \mathrm{Bot}[S, B]$$

The intermediate types A and B are absorbed at each Arrow and Judge boundary. No type annotation mentioning a transient intermediate type appears in the final agent signature.

6. Algebraic Laws and Their Engineering Implications

We collect the key algebraic properties. All proofs appeal only to the three monad laws of M.

6.1 Associativity of Seq

Theorem 6.1. $(\mathrm{Arr}[S], \mathrm{Seq}, \eta_S)$ *is a monoid.*

Proof. Associativity is Lemma 4.1. Left and right identity is Lemma 4.2. □

Engineering implication: agentic behaviour can be grouped and re-grouped arbitrarily. A three-stage agent assembled top-down behaves identically to one assembled bottom-up. Refactoring cannot silently break composition order.

6.2 Identity Erasure

Corollary 6.2. *Optional steps (e.g. a disabled guardrail) can be replaced by η_S without changing agent semantics:*

$$\mathrm{Seq}(f,\ \eta_S,\ g) \;=\; \mathrm{Seq}(f, g)$$

Proof. By Lemma 4.2 (right identity) applied to $\mathrm{Seq}(f, \eta_S) = f$, then substitution. □

6.3 Arrow Coherence

Theorem 6.3. Arrow *maps* $\mathrm{Bot}[S, A]$ *into* $\mathrm{End}_{\mathbf{Kl}(M)}(S)$*, and Kleisli composition of arrows is* Seq*:*

$$\mathrm{Arrow}(b_1, \varphi_1) \gg\!\!> \mathrm{Arrow}(b_2, \varphi_2) \;=\; \mathrm{Seq}(\mathrm{Arrow}(b_1, \varphi_1),\ \mathrm{Arrow}(b_2, \varphi_2))$$

Proof. By definition, Seq is iterated $\gg\!\!>$. For a two-element sequence the expansion is the single Kleisli composition. □

The coherence property means that lifting commutes with sequencing — a desirable condition for any combinator library.

6.4 Map Functor Laws

Theorem 6.4. Map *satisfies the functor laws under* S*-indexed composition* $(\bullet)$*:*

1. $\mathrm{Map}(b, \pi_2) = b$ *(identity — Lemma 3.7)*
2. $\mathrm{Map}(\mathrm{Map}(b, g), f) = \mathrm{Map}(b, f \bullet g)$ *(composition — Lemma 3.8)*

These guarantee that output transformations compose predictably and that $\mathrm{Map}(b, f)$ does not re-execute the bot.

7. Comparison with Existing Frameworks

Framework	Composition primitive	Type safety	Algebraic guarantees
LangChain (Python)	`chain` | operator	Runtime	None stated
LlamaIndex	Pipeline DAG (dict-typed)	Runtime	None stated
AutoGen	Conversational loops	Runtime	None stated
DSPy	Module composition	Partial (Python)	Informal
nanobot (this work)	Kleisli arrow $\mathrm{Arr}[S]$	Compile-time (generics)	Monad laws, functor laws

The key differentiator is **compile-time type safety across composition boundaries**. The intermediate output type A of any bot is *absorbed* into the blackboard by Arrow at construction time; the agent's type signature $\mathrm{Arr}[S]$ does not mention transient intermediate types. No runtime type assertion is required.

8. Discussion

8.1 Blackboard as a First-Class Concept

The design choice to make the blackboard type S explicit in every bot signature — rather than threading implicit context — has algebraic consequences. It makes $\mathrm{Bot}[S, A]$ a genuine functor from the category of state types to the category of effectful computations, enabling the Map combinator and the automatic lens derivation in Arrow.

8.2 Why Three Patterns, Not One

Seq, $\mathrm{Reflect}$, and $\mathrm{ThinkReAct}$ are all built from the same primitive types, but they are categorically distinct:

- Seq is a *monoid* product in $\mathrm{End}_{\mathbf{Kl}(M)}(S)$.
- $\mathrm{Reflect}$ is a *bounded iteration* with a decision guard in $\mathbf{Kl}(M)(S)$.
- $\mathrm{ThinkReAct}$ is a *cross-category traversal* bridging $\mathbf{Kl}(M)(S)$ and $\mathbf{Kl}(M)(T)$ via transform/fold.

Both $\mathrm{Reflect}$ and $\mathrm{ThinkReAct}$ share a common structure: a specialised bot ($\mathrm{Bot}[S, \mathrm{Vote}[S]]$ or $\mathrm{Bot}[S, [T]]$) followed by an arrow tail ($\mathrm{Arr}[S]$ or $\mathrm{Arr}[T]$). The key difference is that $\mathrm{ThinkReAct}$ crosses state spaces — the reactor $\mathrm{Arr}[T]$ operates in the inner per-task category while the overall combinator produces $\mathrm{Arr}[S]$ via the gather fold.

All three produce $\mathrm{Arr}[S]$ (or the isomorphic $\mathrm{Bot}[S, S]$), making them freely composable with each other. The Think combinator provides the bridge between the planner's output type $[A]$ and the reactor's state type T.

8.3 Reflect as a Bounded Fixed Point

The reflection loop is a bounded iteration of the form:

$$s_{i+1} = c((\pi_{\text{State}} \circ j)(s_i))$$

This is not a general fixed point (which would require a complete lattice and a monotone function); it is a *bounded unfolding* of the recurrence, implemented by Kleisli composition inside a structural recursion on the attempt counter (Lemma 4.6). The bound n is an engineering safeguard against non-termination due to a misbehaving judge or corrector.

8.4 Vote and Judge: Separation of Evaluation and Decision

The decomposition of the reflection loop's input into two separate concerns — evaluating content ($\text{Bot}[S, A]$) and rendering a verdict ($\text{Eff}[\text{Vote}[S], A]$) — via the Judge combinator is a consequence of the algebra. The raw judge bot need not know about the Vote type or the accept/reject protocol. The effect $\varphi : \text{Eff}[\text{Vote}[S], A]$ encodes the decision policy separately, promoting reuse: the same underlying evaluator bot can serve different reflection policies by varying only φ.

8.5 Hoist: Cross-Type Pipeline Reuse

Arrow and Lift both embed computations *into* $\text{Arr}[S]$ from below — from $\text{Bot}[S, A]$ or from $\text{Eval}[S]$. Hoist embeds from the *side*: it takes a finished $\text{Arr}[T]$ pipeline and integrates it into $\text{Arr}[S]$ without any knowledge of T's internal structure beyond two lens fields.

This is architecturally significant. A team may build a complete $\text{Arr}[T]$ pipeline for a focused sub-task (e.g. entity extraction, document classification) and then embed it into a larger orchestration blackboard S by providing only the input/output field mapping $\text{BiMap}[S, A, T, B]$. The inner pipeline does not need to be rewritten; the outer pipeline does not need to know its internal structure.

The inject/project steps in Hoist (Lemma 3.10) ensure that the outer blackboard is *surgically* updated: only the designated output field B changes. This is the cross-type analogue of the lens-based put operation that Arrow uses within a single type.

8.6 ReAct and Pure: Two Modes of Bot Construction

Every combinator in the hierarchy except ReAct is a *deterministic* structural transformation: Arrow applies a pure fold and a side-effect; Seq threads state; Reflect iterates; Think scatters; and ThinkReAct traverses and gathers. The only source of non-determinism (and the only component that contacts an external LLM) is ReAct. Pure is the deterministic counterpart: it lifts an ordinary function into $\text{Bot}[S, A]$ without invoking a model. Together, ReAct and Pure exhaust the two construction modes for Bot. This separation has a practical consequence: the structural combinators can be tested and verified independently of any language model, and the composition laws (Theorems 6.1–6.4) hold regardless of which primitive produced the Bot leaves.

9. Conclusion

We have shown that the behaviour of LLM agents over a shared blackboard is faithfully modelled by the Kleisli category of the error-state monad. The three primitive types — $\text{Eval}[S]$ (effectful state transformer), $\text{Lens}[S, A]$ (lens put), and $\text{Eff}[S, A]$ (their product) — together with the Arrow lift and the $\text{Arr}[S]$ type, form a minimal and complete algebra for agent step construction.

The two primitive realisations of Bot are ReAct (non-deterministic, LLM-backed) and Pure (deterministic, no model call). ReAct is the sole source of non-determinism; Pure embeds ordinary computations — field projections, derived values, sub-list extraction — into the same $\text{Bot}[S, A]$ interface. Every other combinator is a deterministic structural transformation.

The three structural combinators — Seq (monoid composition), Reflect (bounded iteration with Vote/Judge), and ThinkReAct (cross-category transform/unfold traversal) — are derivable instances of standard categorical constructions. Lift, Hoist, Judge, Think, and Map are derived forms that improve ergonomics without extending the core. In particular,

Hoist (§3.8) introduces a lens-based partial isomorphism $\mathrm{BiMap}[S, A, T, B]$ that lifts any $\mathrm{Arr}[T]$ into $\mathrm{Arr}[S]$, enabling type-safe reuse of sub-pipelines across differently-shaped blackboards with surgical field-level coupling (Lemmas 3.9–3.10).

All key properties — associativity of Seq (Theorem 6.1), identity erasure (Corollary 6.2), functor laws for Map (Theorem 6.4), and termination of $\mathrm{Reflect}$ (Lemma 4.6) — have been proved from the monad laws alone.

References

1. Moggi, E. (1991). *Notions of computation and monads.* Information and Computation, 93(1), 55–92.
2. Wadler, P. (1995). *Monads for functional programming.* In Advanced Functional Programming, LNCS 925.
3. Pierce, B. C. (1991). *Basic Category Theory for Computer Scientists.* MIT Press.
4. Foster, J. N., Greenwald, M. B., Moore, J. T., Pierce, B. C., & Schmitt, A. (2007). *Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem.* ACM TOPLAS, 29(3).
5. Kmett, E. (2012). *lens: Lenses, Folds and Traversals.* `hackage.haskell.org/package/lens`.
6. Kolesnikov, D. (2025–2026). *thinker: Type-safe LLM agent framework.* `github.com/kshard/thinker`.